

AdoptaSV

Documentación Técnica del Backend

API Laravel · Firebase Auth · PostgreSQL

Versión 1.0 — Junio 2026

1. Autenticación Firebase + API

1.1 Visión General

La API no gestiona contraseñas directamente. La autenticación se delega completamente a Firebase Authentication. El servidor solo verifica que el token JWT emitido por Firebase sea válido, sin almacenar ni procesar credenciales.

Flujo de Autenticación

El proceso completo de autenticación sigue estos pasos:

- El cliente (Android/iOS/web) autentica al usuario usando Firebase Authentication con cualquier proveedor (email/password, Google, etc.).
- Firebase emite un ID Token firmado (JWT RSA-256).
- El cliente envía el token en cada request a la API como Bearer Token en el header Authorization.
- El middleware firebase.auth en el servidor verifica el token contra Firebase SDK.
- El servidor busca o crea el usuario en la base de datos local.
- La respuesta continúa normalmente si el token es válido y la cuenta está activa.

1.2 Configuración del Servidor

1.2.1 Archivo de Credenciales (Service Account)

El SDK de Firebase para PHP necesita un Service Account JSON para autenticar las llamadas a la API de Google.

- Ir a Firebase Console → Proyecto → Configuración → Cuentas de servicio.
- Generar nueva clave privada → descargar el archivo .json.
- Colocar el archivo en: storage/firebase/service-account.json

1.2.2 Variables de Entorno Requeridas

```
FIREBASE_CREDENTIALS=storage/firebase/service-account.json
FIREBASE_PROJECT_ID=mi-proyecto-firebase
FIREBASE_STORAGE_DEFAULT_BUCKET=mi-proyecto-firebase.appspot.com
FIREBASE_DATABASE_URL=https://mi-proyecto-firebase-default-rtdb.firebaseio.com
```

Variable	Descripción	Ejemplo
FIREBASE_CREDENTIALS	Ruta al Service Account JSON	storage/firebase/service-account.json
FIREBASE_PROJECT_ID	ID del proyecto Firebase	adopta-sv-prod
FIREBASE_STORAGE_DEFAULT_BUCKET	Bucket de Firebase Storage	adopta-sv-prod.appspot.com
FIREBASE_DATABASE_URL	URL de Realtime Database (opcional)	https://adopta-sv-prod-default-rtdb.firebaseio.com
FIREBASE_AUTH_TENANT_ID	Tenant ID para multi-tenancy (no activo)	—
FIREBASE_CACHE_STORE	Driver de cache para claves públicas	file (default)
FIREBASE_HTTP_LOG_CHANNEL	Canal de logs HTTP de Firebase (debug)	daily

1.2.3 Configuración (config/firebase.php)

```
'default' => env('FIREBASE_PROJECT', 'app'),

'projects' => [
    'app' => [
        'credentials' => env('FIREBASE_CREDENTIALS'),
        'auth' => ['tenant_id' => env('FIREBASE_AUTH_TENANT_ID')],
        'storage' => ['default_bucket' => env('FIREBASE_STORAGE_DEFAULT_BUCKET')],
```

```
'cache_store' => env('FIREBASE_CACHE_STORE', 'file'),  
],  
],
```

cache_store: file — las claves públicas de Firebase (usadas para verificar tokens) se cachean en disco para evitar llamadas repetidas a Google.

tenant_id — solo necesario si se usa Firebase Multi-tenancy (no activo en este proyecto).

1.3 Middleware FirebaseAuthenticate

Archivo: app/Http/Middleware/FirebaseAuthenticate.php — Alias de ruta: firebase.auth

Código del Middleware — Lógica Paso a Paso

```
public function handle(Request $request, Closure $next)  
{  
    // 1. Extraer el Bearer Token del header Authorization  
    $token = $request->bearerToken();  
  
    if (!$token) {  
        return response()->json(['message' => 'Token no proporcionado.'], 401);  
    }  
  
    // 2. Verificar el token con Firebase SDK  
    // - Valida la firma criptográfica (RSA-256)  
    // - Verifica que no haya expirado (exp claim)  
    // - Verifica que el issuer sea Firebase  
    // - El segundo argumento 'true' verifica tokens revocados  
    try {  
        $verifiedToken = $this->auth->verifyIdToken($token, true);  
    } catch (FailedToVerifyToken $e) {  
        return response()->json(['message' => 'Token inválido o expirado.'], 401);  
    }  
}
```

```
}

// 3. Extraer UID y email del payload del JWT
$uid = $verifiedToken->claims()->get('sub'); // subject = Firebase UID
$email = $verifiedToken->claims()->get('email', $uid . '@firebase.local');

// 4. Buscar usuario por UID, luego por email como fallback
$user = User::where('firebase_uid', $uid)->first()
    ?? User::where('email', $email)->first();

// 5. Si no existe → auto-registro con rol adoptante
if (!$user) {
    $user = User::create([
        'firebase_uid' => $uid,
        'name'         => $verifiedToken->claims()->get('name', 'Sin nombre'),
        'email'         => $email,
        'estado_cuenta' => EstadoCuenta::Activo,
    ]);
    $user->assignRole('adoptante');

// 6. Si existe pero el UID no coincide → actualizar UID (migración de cuenta)
} elseif ($user->firebase_uid !== $uid) {
    $user->update(['firebase_uid' => $uid]);
}

// 7. Verificar que la cuenta esté activa
if ($user->estado_cuenta !== EstadoCuenta::Activo) {
    return response()->json(['message' => 'Cuenta suspendida o inactiva.'], 403);
}

// 8. Inyectar usuario en el request
$request->merge(['firebase_uid' => $uid]);
$request->setUserResolver(fn () => $user);
auth()->setUser($user);
```

```
return $next($request);
}
```

Verificación de Revocación

El segundo parámetro true en `verifyIdToken($token, true)` activa la verificación contra tokens revocados. Cuando se llama a `logout → auth->revokeRefreshTokens($uid)`, los nuevos requests con el token anterior retornarán 401 aunque el JWT no haya expirado técnicamente.

1.4 Auto-registro de Usuarios

El sistema crea automáticamente un usuario en la base de datos local la primera vez que se autentica con Firebase. No hay endpoint de registro separado.

Claims del JWT Usados en el Registro

Claim JWT	Campo en users	Fallback
sub	firebase_uid	— (siempre presente)
email	email	{uid}@firebase.local
name	name	'Sin nombre'

Campos NO Tomados del Token

Campo	Valor asignado
password	null (no se usa con Firebase)
estado_cuenta	activo (por defecto)
telefono, foto_perfil_url	direccion, null (el usuario los completa después via PUT /api/perfil)

Escenario: Usuario ya existía por Email

Si el usuario fue creado previamente (ej: seeder de prueba) con el mismo email pero sin `firebase_uid`, el middleware lo detecta por email y le asigna el `firebase_uid` del token. Esto evita duplicados cuando se migran datos.

1.5 Sistema de Roles

Los roles se gestionan con Spatie Permission y se asignan desde el middleware o desde UserController.

Rol	Asignado por	Descripción
adoptante	Auto (middleware)	Rol por defecto de todo usuario nuevo
voluntario	Admin del refugio (al aprobar solicitud)	Puede gestionar refugios asignados
admin	Solo via PATCH <code>/api/users/{id}/role</code>	Acceso total al sistema

Cómo se Asciende de Rol

```

Usuario nuevo ————— rol: adoptante
|
| POST /api/refugios/{refugio}/solicitudes-voluntario
▼
SolicitudVoluntario (estado: pendiente)
|
| PATCH /api/solicitudes-voluntario/{id}/estado { "estado": "aprobada" }
▼
assignRole('voluntario') + attach a refugio_voluntario — rol: voluntario
|
| PATCH /api/users/{id}/role { "role": "admin" } (solo otro admin)
▼
syncRoles(['admin']) ————— rol: admin
  
```

1.6 Endpoints de Autenticación

GET /api/auth/me

Retorna la identidad del usuario autenticado con sus roles y permisos. Útil para que el cliente sepa qué puede hacer.

```
GET /api/auth/me
Authorization: Bearer <firebase_id_token>

// Response 200:
{
  "id": 1,
  "firebase_uid": "VhF3kQm9ZTNx...",
  "name": "María García",
  "email": "maria@example.com",
  "roles": ["admin"],
  "permissions": ["manage users", "manage shelters"]
}
```

POST /api/auth/logout

Revoca todos los refresh tokens del usuario en Firebase. Después de esto, el cliente debe volver a hacer login para obtener un nuevo ID Token.

```
POST /api/auth/logout
Authorization: Bearer <firebase_id_token>

// Response 200:
{ "message": "Sesión cerrada correctamente." }
```


1.7 Uso del Token desde el Cliente

Android — Kotlin + Retrofit (Interceptor)

```
class FirebaseAuthInterceptor : Interceptor {

    override fun intercept(chain: Interceptor.Chain): Response {

        val user = FirebaseAuth.getInstance().currentUser

        ?: return chain.proceed(chain.request())

        val token = runBlocking {

            Tasks.await(user.getIdToken(false)).token

        } ?: return chain.proceed(chain.request())

        val request = chain.request().newBuilder()

            .addHeader("Authorization", "Bearer $token")

            .addHeader("Content-Type", "application/json")

            .build()

        return chain.proceed(request)

    }

}

// Configuración de Retrofit
val retrofit = Retrofit.Builder()

    .baseUrl("https://api.adoptasv.com/api/")

    .addConverterFactory(GsonConverterFactory.create())

    .client(

        OkHttpClient.Builder()

            .addInterceptor(FirebaseAuthInterceptor())

            .build()

    )

    .build()
```

CURL — Pruebas

1. Obtener token via Firebase REST API

```
TOKEN=$(curl -s -X POST \
  "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=<API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123","returnSecureToken":true}' \
  | jq -r '.idToken')
```

2. Usar el token en la API

```
curl -H "Authorization: Bearer $TOKEN" \
  https://api.adoptasv.com/api/auth/me
```

1.8 Errores y Códigos de Respuesta

Errores del Middleware

Situación	HTTP	Body JSON
Header Authorization ausente	401	{"message": "Token no proporcionado."}
Token con firma inválida	401	{"message": "Token inválido o expirado."}
Token expirado (exp en el pasado)	401	{"message": "Token inválido o expirado."}
Token revocado (después de logout)	401	{"message": "Token inválido o expirado."}
Token de proyecto Firebase diferente	401	{"message": "Token inválido o expirado."}
Cuenta con estado_cuenta = inactivo	403	{"message": "Cuenta suspendida o inactiva."}
Cuenta con estado_cuenta = suspendido	403	{"message": "Cuenta suspendida o inactiva."}

Errores de Roles

Situación	HTTP	Generado por
Rol insuficiente para el endpoint	403	Spatie RoleMiddleware
No es propietario del recurso	403	abort_unless() en el controller

1.9 Tests de la Capa de Auth

Archivo: tests/Feature/FirebaseAuthMiddlewareTest.php

Los tests mockean el SDK de Firebase para evitar llamadas reales a Google durante las pruebas.

Setup del Mock

```
protected function setUp(): void
{
    parent::setUp();

    // Sustituye la implementación real de FirebaseAuth
    $this->mockAuth = Mockery::mock(FirebaseAuth::class);
    $this->app->instance(FirebaseAuth::class, $this->mockAuth);
}
```

Casos de Prueba Cubiertos

Test	Escenario	Resultado esperado
test_request_sin_token_retorna_401	Request sin header Authorization	401 + "Token no proporcionado."
test_token_invalido_retorna_401	SDK lanza FailedToVerifyToken	401 + "Token inválido o expirado."

Test	Escenario	Resultado esperado
test_token_valido_crea_usuario_nuevo_y_retorna_200	UID nuevo, usuario no existe en BD	200, usuario creado en BD
test_token_valido_usuario_existente_retorna_200	UID ya existe en BD	200, no crea duplicado
test_cuenta_suspendida_retorna_403	estado_cuenta = suspendido	403 + "Cuenta suspendida o inactiva."
test_cuenta_inactiva_retorna_403	estado_cuenta = inactivo	403 + "Cuenta suspendida o inactiva."
test_no_crea_usuario_duplicado_en_requests_sucesivos	Dos requests con el mismo UID	Solo 1 fila en BD

```
// Cómo ejecutar los tests
```

```
php artisan test --filter FirebaseAuthMiddlewareTest
```

1.10 Registro de Middleware en Laravel

Archivo: bootstrap/app.php

```
return Application::configure(basePath: dirname(__DIR__))
    ->withProviders([
        \App\Providers\FirebaseStorageServiceProvider::class,
    ])
    ->withRouting(
        api: __DIR__.'/../routes/api.php',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'firebase.auth' => \App\Http\Middleware\FirebaseAuthenticate::class,
            'role'          => \Spatie\Permission\Middleware\RoleMiddleware::class,
```

```
'permission' => \Spatie\Permission\Middleware\PermissionMiddleware::class,
'role_or_permission' => \Spatie\Permission\Middleware\RoleOrPermissionMiddleware::class,
]);
})
->withExceptions(function (Exceptions $exceptions): void {
    $exceptions->shouldRenderJsonWhen(
        fn (Request $request) => $request->is('api/*')
    );
})->create();
```

Aplicación en routes/api.php

```
// Rutas públicas (sin autenticación)
Route::get('/health', fn () => response()->json(['status' => 'ok']));
Route::get('/files/{path}', [ServeFileController::class, 'serve']->where('path', '!+'));

// Todas las demás rutas requieren token Firebase
Route::middleware('firebase.auth')->group(function () {

    Route::get('/auth/me', [AuthController::class, 'me']);
    Route::post('/auth/logout', [AuthController::class, 'logout']);

    // Rutas con restricción de rol
    Route::post('/mascotas', [MascotaController::class, 'store'])
        ->middleware('role:admin|voluntario');

    Route::prefix('users')->middleware('role:admin')->group(function () {
        Route::patch('/{id}/role', [UserController::class, 'assignRole']);
    });
});
```

1.11 Estructura del Service Account

```
{
  "type": "service_account",
  "project_id": "adopta-sv-prod",
  "private_key_id": "abc123...",
  "private_key": "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA PRIVATE KEY-----\n",
  "client_email": "firebase-adminsdk-xxx@adopta-sv-prod.iam.gserviceaccount.com",
  "client_id": "123456789",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/..."
}
```

Seguridad del Archivo

- No commitear en el repositorio. Agregar storage/firebase/ a .gitignore
- En producción (Railway, etc.) se puede pasar como variable de entorno codificada en Base64.
- El SDK de kreait/laravel-firebase acepta tanto ruta de archivo como JSON inline.

Permisos Mínimos en el Service Account

Permiso Firebase	Para qué
Firebase Authentication Admin	Verificar ID Tokens, revocar refresh tokens
Firebase Storage Admin	Subir/eliminar archivos (si se usa FirebaseStorageService)

1.12 Resumen de Seguridad

Aspecto	Implementación
Autenticación	Firebase ID Token (JWT RSA-256, firmado por Google)
Verificación de revocación	Activada (verifyIdToken(\$token, true))
Contraseñas en BD	No se almacenan (password = null con Firebase Auth)
Expiración de token	1 hora (gestionada por Firebase SDK del cliente)
Path traversal en archivos	Bloqueado por ServeFileController (abort_if str_contains)
Archivos públicos	URLs con UUID — imposibles de adivinar, no requieren auth
Cuentas suspendidas	Bloqueadas en el middleware antes de procesar cualquier request
Cambio de rol propio	Bloqueado en UserController (abort_if \$request->user()->id === \$id)

2. Controllers, Requests y Services

Base URL: /api — Auth: todos los endpoints (excepto /health y /files/{path}) requieren Authorization:
Bearer <firebase_id_token>

2.1 AuthController

Archivo: app/Http/Controllers/Api/AuthController.php — Dependencia: Kreait\Firebase\Contract\Auth

Método	Ruta	Rol requerido	Descripción
GET	/api/auth/me	Autenticado	Retorna info del usuario autenticado
POST	/api/auth/logout	Autenticado	Revoca los refresh tokens en Firebase

2.2 PerfilController

Archivo: app/Http/Controllers/Api/PerfilController.php

Método	Ruta	Rol requerido	Descripción
GET	/api/perfil	Autenticado	Ver perfil propio (incluye relación refugio)
PUT	/api/perfil	Autenticado	Actualizar perfil propio

Campos Permitidos en PUT /api/perfil

Campo	Tipo	Validación
name	string	max:150
telefono	string null	max:20
direccion	string null	max:255
foto_perfil_url	string null	url

2.3 RefugioController

Archivo: app/Http/Controllers/Api/RefugioController.php — Request: StoreRefugioRequest — Resource: RefugioResource

Método	Ruta	Rol requerido	Descripción
GET	/api/refugios	Autenticado	Listar refugios activos
POST	/api/refugios	admin, voluntario	Crear refugio
GET	/api/refugios/{refugio}	Autenticado	Ver refugio + mascotas
PUT	/api/refugios/{refugio}	admin, voluntario	Editar refugio
DELETE	/api/refugios/{refugio}	admin, voluntario	Desactivar refugio (soft-delete)
GET	/api/refugios/{refugio}/voluntarios	admin, voluntario	Listar voluntarios del refugio

Método	Ruta	Rol requerido	Descripción
POST	/api/refugios/{refugio}/voluntarios	admin, voluntario	Agregar voluntario
DELETE	/api/refugios/{refugio}/voluntarios/{user}	admin, voluntario	Remover voluntario de tabla pivot

Comportamientos Importantes

- GET /api/refugios — Retorna solo refugios con activo = true.
- POST /api/refugios — Valida que el usuario no tenga ya un refugio (422 si existe). Asigna user_id automáticamente. Respuesta 201.
- PUT /api/refugios/{refugio} — Solo el propietario (user_id) o un admin puede editar. 403 si no tiene permisos.
- DELETE /api/refugios/{refugio} — Soft-delete lógico: cambia activo = false, no elimina el registro.
- POST /api/refugios/{refugio}/voluntarios — Verifica que el usuario no sea ya voluntario. Asigna automáticamente el rol voluntario si no lo tiene.
- DELETE /api/refugios/{refugio}/voluntarios/{user} — Solo remueve de la tabla pivot; no elimina el rol globalmente.

2.4 MascotaController

Archivo: app/Http/Controllers/Api/MascotaController.php — Request: StoreMascotaRequest — Resource: MascotaResource

Método	Ruta	Rol requerido	Descripción
GET	/api/mascotas	Autenticado	Listar mascotas disponibles (paginado 20)
POST	/api/mascotas	admin, voluntario	Registrar mascota
GET	/api/mascotas/{mascota}	Autenticado	Ver mascota + refugio
PUT	/api/mascotas/{mascota}	admin, voluntario	Editar mascota
DELETE	/api/mascotas/{mascota}	admin, voluntario	Eliminar mascota

Filtros GET /api/mascotas (Query Params)

Param	Tipo	Valores
especie	string	perro, gato
sexo	string	macho, hembra
tamaño	string	pequeno, mediano, grande
refugio_id	integer	ID del refugio

Lógica de Asignación de Refugio en POST /api/mascotas

- 1. Usa refugio_id del body si viene.
- 2. Si no viene, usa el refugio propio del usuario (user->refugio).
- 3. Si tampoco tiene refugio propio, usa el primer refugio donde es voluntario.
- 4. Si ninguno aplica → 403.

PUT /api/mascotas/{mascota} — Excluye refugio_id del update (no se puede cambiar el refugio de una mascota). Solo admin o quien gestione el refugio de la mascota.

2.5 SolicitudController

Archivo: app/Http/Controllers/Api/SolicitudController.php — Request: StoreSolicitudRequest — Dependencia: ScoringService

Método	Ruta	Rol requerido	Descripción
GET	/api/solicitudes/mis-solicitudes	Autenticado	Ver solicitudes propias
POST	/api/solicitudes	Autenticado	Enviar solicitud de adopción
GET	/api/solicitudes/{solicitud}	Autenticado	Ver solicitud (adoptante o refugio)

Método	Ruta	Rol requerido	Descripción
GET	/api/solicitudes/{solicitud}/seguimientos	Autenticado	Ver seguimientos de una adopción
GET	/api/solicitudes	admin, voluntario	Listar solicitudes del refugio
PATCH	/api/solicitudes/{solicitud}/estado	admin, voluntario	Cambiar estado

Validaciones de Negocio al Crear Solicitud

- La mascota debe estar en estado disponible.
- El usuario no debe tener ya una solicitud pendiente o en_revision para esa mascota.
- Al crear: genera la solicitud con estado pendiente, cambia la mascota a en_proceso, llama a ScoringService::calcularYGuardar().

PATCH /api/solicitudes/{solicitud}/estado

```
{
  "estado": "aprobada",
  "comentario": "Todo en orden, bienvenido!"
}
```

Estado destino	Acción
en_revision	Actualiza estado a en_revision
aprobada	Llama a Solicitud::aprobar() → mascota pasa a adoptada, demás solicitudes se rechazan automáticamente
rechazada	Llama a Solicitud::rechazar() → mascota vuelve a disponible

2.6 SeguimientoController

Archivo: app/Http/Controllers/Api/SeguimientoController.php — Request: StoreSeguimientoRequest — Resource: SeguimientoResource

Método	Ruta	Rol requerido	Descripción
GET	/api/seguimientos/mis-seguimientos	Autenticado	Ver mis seguimientos enviados
POST	/api/seguimientos	Autenticado	Enviar reporte de seguimiento
GET	/api/seguimientos/{seguimiento}	Autenticado	Ver detalle
PATCH	/api/seguimientos/{seguimiento}/observacion	admin, voluntario	Agregar observación del refugio

Validaciones POST /api/seguimientos

- El solicitud_id debe pertenecer al usuario autenticado.
- La solicitud debe estar en estado aprobada.
- mascota_id se toma automáticamente de la solicitud (no se envía en el body).

PATCH /api/seguimientos/{seguimiento}/observacion

```
{
  "observacion_refugio": "La mascota se ve saludable.",
  "estado_seguimiento": "revisado"
}
```

estado_seguimiento es opcional; default: revisado. Solo admin o quien gestione el refugio de la solicitud.

2.7 ReporteController

Archivo: app/Http/Controllers/Api/ReporteController.php — Request: StoreReporteRequest — Resource: ReporteResource

Método	Ruta	Rol requerido	Descripción
GET	/api/reportes/mis-reportes	Autenticado	Ver reportes propios (paginado)
GET	/api/reportes	Autenticado	Listar todos los reportes (paginado)
POST	/api/reportes	Autenticado	Crear reporte SOS
GET	/api/reportes/{reporte}	Autenticado	Ver reporte
PATCH	/api/reportes/{reporte}/estado	admin, voluntario	Cambiar estado del reporte
POST	/api/reportes/{reporte}/convertir-mas-cota	admin, voluntario	Convertir reporte en mascota del refugio

POST /api/reportes/{reporte}/convertir-mascota

Convierte un reporte SOS en una mascota registrada en un refugio. Automáticamente cambia el estado del reporte a atendido.

```
{
  "refugio_id": 2,
  "nombre": "Luna",
  "especie": "perro",
  "sexo": "hembra",
  "raza": "mestiza",
  "edad_meses": 6
}

// Respuesta 201: MascotaResource de la mascota creada.
```

2.8 PanelRefugioController

Archivo: app/Http/Controllers/Api/PanelRefugioController.php — Middleware: role:admin|voluntario

Panel de gestión para administradores y voluntarios de un refugio.

Método	Ruta	Descripción
GET	/api/panel/resumen	Estadísticas del refugio
GET	/api/panel/mascotas	Todas las mascotas del refugio
PATCH	/api/panel/mascotas/{mascota}/estado	Cambiar estado de mascota
GET	/api/panel/mascotas/{mascota}/solicitudes	Solicitudes de una mascota específica
GET	/api/panel/solicitudes	Todas las solicitudes del refugio
GET	/api/panel/seguimientos	Seguimientos pendientes del refugio

Resolución de Refugio (Helper Privado)

- Si se envía ?refugio_id=X en el query → usa ese refugio (verifica que el user lo gestione).
- Si el user gestiona un solo refugio → lo usa automáticamente.
- Si gestiona más de uno → exige refugio_id (422 si no se envía).

GET /api/panel/resumen — Respuesta

```
{
  "mascotas": {
    "disponibles": 12,
    "en_proceso": 3,
    "adoptadas": 45
  },
  "solicitudes": {
    "pendientes": 5,
    "en_revision": 2,
```

```
"aprobadas": 30
},
"seguimientos_pendientes": 4
}
```

2.9 FotoController

Archivo: app/Http/Controllers/Api/FotoController.php — Dependencia: LocalStorageService

Gestión de subida y eliminación de imágenes. Todas las fotos se almacenan en el disco public y se sirven a través de /api/files/{path}.

Método	Ruta	Campo	Max	Descripción
POST	/api/perfil/foto	foto	2 MB	Foto de perfil del usuario
POST	/api/mascotas/{mascota}/foto	foto	5 MB	Foto principal de mascota
POST	/api/mascotas/{mascota}/fotos-extra	foto	5 MB	Agregar foto extra a mascota
DELETE	/api/mascotas/{mascota}/fotos-extra	foto_url	—	Eliminar foto extra específica
POST	/api/reportes/{reporte}/foto	foto	5 MB	Foto del reporte SOS
POST	/api/refugios/{refugio}/logo	logo	2 MB	Logo del refugio
POST	/api/seguimientos/{seguimiento}/foto	foto	5 MB	Foto del seguimiento

Formatos permitidos: jpg, jpeg, png, webp

Para foto principal (perfil, mascota, reporte, refugio, seguimiento): si ya existe una foto anterior, la elimina del almacenamiento antes de subir la nueva.

2.10 MapaController

Archivo: `app/Http/Controllers/Api/MapaController.php` — Endpoints geoespaciales para el mapa de la app. Usan la fórmula de Haversine para calcular distancias.

Método	Ruta	Descripción
GET	<code>/api/mapa/refugios</code>	Refugios activos con filtro por radio
GET	<code>/api/mapa/reportes</code>	Reportes SOS activos con filtro por radio (excluye cerrados)

Query Params Comunes

Param	Tipo	Descripción	Default
lat	numeric	Latitud del usuario (-90 a 90)	—
lng	numeric	Longitud del usuario (-180 a 180)	—
radio	numeric	Radio en km (1-100)	20

Fórmula Haversine (SQL)

```
6371 * ACOS(
  COS(RADIANS(:lat)) * COS(RADIANS(latitud)) *
  COS(RADIANS(longitud) - RADIANS(:lng)) +
  SIN(RADIANS(:lat)) * SIN(RADIANS(latitud))
) AS distancia_km
-- Compatible con PosgreSQL, PostgreSQL y SQLite.
```

Si no se envían lat y lng → retorna todos los registros con coordenadas. Si se envían → filtra por radio y ordena por distancia_km ASC.

2.11 SolicitudVoluntarioController

Archivo: `app/Http/Controllers/Api/SolicitudVoluntarioController.php` — Resource: `SolicitudVoluntarioResource`

Método	Ruta	Rol requerido	Descripción
POST	<code>/api/refugios/{refugio}/solicitudes-voluntario</code>	Autenticado	Postularse como voluntario
GET	<code>/api/refugios/{refugio}/solicitudes-voluntario</code>	admin, voluntario	Ver postulaciones del refugio
GET	<code>/api/voluntarios/mis-solicitudes</code>	Autenticado	Ver mis postulaciones
PATCH	<code>/api/solicitudes-voluntario/{solicitud}/estado</code>	admin, voluntario	Aprobar o rechazar postulación

PATCH `/api/solicitudes-voluntario/{solicitud}/estado`

```
{
  "estado": "aprobada",
  "comentario": "Bienvenido al equipo."
}
```

- Al aprobar: asigna rol voluntario al solicitante (si no lo tiene), agrega al usuario en tabla pivot `refugio_voluntario` con `syncWithoutDetaching`.
- Al rechazar: solo actualiza el estado y comentario.

2.12 UserController

Archivo: app/Http/Controllers/Api/UserController.php — Middleware: role:admin — Administración de usuarios, solo accesible para admins.

Método	Ruta	Descripción
GET	/api/users	Listar todos los usuarios (paginado, 20)
GET	/api/users/{id}	Ver usuario por ID
PATCH	/api/users/{id}/role	Asignar rol a un usuario (syncRoles — reemplaza todos los roles)
PATCH	/api/users/{id}/estado	Cambiar estado de cuenta (activo/inactivo/suspendido)

⚠ El admin no puede cambiar su propio rol ni su propio estado de cuenta (403 bloqueado en el controller).

2.13 ServeFileController

Archivo: app/Http/Controllers/Api/ServeFileController.php

Método	Ruta	Auth	Descripción
GET	/api/files/{path}	No	Servir archivo del disco público

- Protegido contra path traversal (.. en la ruta → 400).
- Devuelve 404 si el archivo no existe en el disco public.
- Envía el Content-Type real del archivo.
- Header Cache-Control: private, max-age=3600.
- Las URLs contienen UUID, lo que hace que sean prácticamente imposibles de adivinar.

2.14 Form Requests

StoreRefugioRequest — POST/PUT /api/refugios

Campo	Reglas	Notas
nombre	required, string, max:150	
direccion	required, string, max:255	
telefono	nullable, string, max:20	
correo	nullable, email	
descripcion	nullable, string	
logo_url	nullable, string, url	
latitud	nullable, numeric, between:-90,90	
longitud	nullable, numeric, between:-180,180	

StoreMascotaRequest — POST/PUT /api/mascotas

Campo	Reglas	Notas
refugio_id	nullable, integer, exists:refugios	Opcional en POST; ignorado en PUT
nombre	required, string, max:100	
especie	required, Enum(Especie)	perro, gato
raza	nullable, string, max:100	
edad_meses	nullable, integer, min:0, max:240	
sexo	required, Enum(Sexo)	macho, hembra
tamaño	nullable, Enum(Tamano)	pequeno, mediano, grande
descripcion	nullable, string	
personalidad	nullable, string, max:255	
estado_salud	nullable, string, max:255	
vacunas	boolean	

Campo	Reglas	Notas
esterilizado	boolean	
foto_url	nullable, string, url	
fotos_extra	nullable, array	
fotos_extra.*	string, url	Cada elemento del array

StoreSolicitudRequest — POST /api/solicitudes

Campo	Reglas	Notas
mascota_id	required, exists:mascotas	
respuestas_formulario	required, array	Acepta alias datos_adicionales (compatibilidad Android)
respuestas_formulario.tipo_vivien da	required, string	casa, apartamento, etc.
respuestas_formulario.tiene_pat io	required, boolean	
respuestas_formulario.otros_an i males	required, boolean	
respuestas_formulario.experienc ia	required, string	
respuestas_formulario.horas_en _casa	required, integer, min:0, max:24	
respuestas_formulario.comprom iso	required, string	

StoreReporteRequest — POST /api/reportes

Campo	Reglas	Notas
descripcion	required, string	
foto_url	nullable, string, url	
latitud	nullable, numeric, between:-90,90	
longitud	nullable, numeric, between:-180,180	
direccion_referencia	nullable, string, max:255	

StoreSeguimientoRequest — POST /api/seguimientos

Campo	Reglas	Notas
solicitud_id	required, exists:solicitudes	
foto_url	nullable, string, url, required_without:comentario	
comentario	nullable, string, required_without:foto_url	
estado_mascota	nullable, string, max:100	Descripción libre del estado de salud

2.15 Services

ScoringService — app/Services/ScoringService.php

Inyectado en: SolicitudController. Calcula un puntaje de 0 a 100 para las solicitudes de adopción basado en las respuestas del formulario.

Criterio	Puntos
Tipo de vivienda: casa	25
Tipo de vivienda: apartamento	15
Tipo de vivienda: otro	5
Tiene patio/jardín	+20
Sin otros animales O tiene experiencia (desc > 20 chars)	+15
Tiene experiencia previa documentada	+20
≥ 8 horas en casa por día	+20
≥ 6 horas en casa por día	+15
≥ 4 horas en casa por día	+8
< 4 horas en casa por día	0

Puntaje máximo: 100 (capped con `min($score, 100)`).

Métodos disponibles

- `calcular(Solicitud $solicitud): int` — Calcula y retorna el puntaje sin persistir.
- `calcularYGuardar(Solicitud $solicitud): int` — Calcula el puntaje y lo guarda en `solicitud.puntaje_evaluacion`. Se llama automáticamente al crear una solicitud.

LocalStorageService — app/Services/LocalStorageService.php

Inyectado en: FotoController — Disco: public (Laravel Storage). Servicio activo para gestión de archivos locales. Almacena en `storage/app/public/` y sirve los archivos a través de `ServeFileController`.

`upload(UploadedFile $file, string $folder): string`

- `$file` — Archivo recibido del request
- `$folder` — Carpeta destino: mascotas/5, reportes/12, perfiles/1, etc.
- Genera nombre con UUID: `{folder}/{uuid}.{ext}`
- Retorna URL pública: `{APP_URL}/api/files/{path}`

`delete(string $publicUrl): void`

- Extrae la ruta relativa desde la URL pública (`/api/files/` → ruta local).
- Elimina el archivo del disco si existe.
- No lanza error si el archivo no existe.

Estructura de Almacenamiento

```
storage/app/public/  
mascotas/{id}/  
  {uuid}.jpg      ← foto principal  
  extra/{uuid}.jpg ← fotos extra  
reportes/{id}/{uuid}.jpg  
perfiles/{id}/{uuid}.jpg  
refugios/{id}/{uuid}.jpg  
seguimientos/{id}/{uuid}.jpg
```

FirestoreStorageService — `app/Services/FirestoreStorageService.php`

Estado: Implementado pero no activo (el proyecto usa `LocalStorageService`) — Dependencia: `Kreait\Firebase\Contract\Storage`

Alternativa para almacenar archivos en Firebase Storage (GCS). Disponible para migrar si se requiere almacenamiento en la nube.

- `upload()` — Sube con ACL `publicRead`. Retorna URL pública de Firebase Storage.
- `delete()` — Extrae el `objectName` de la URL de Firebase Storage. Ignora errores si el objeto no existe.

```
// Formato de URL pública:
```

```
https://firebasestorage.googleapis.com/v0/b/{bucket}/o/{encoded_path}?alt=media
```

2.16 API Resources

Los Resources transforman los modelos Eloquent al formato JSON de la API.

Resource	Modelo	Usado en
UserResource	User	AuthController, RefugioController, UserController
RefugioResource	Refugio	RefugioController
MascotaResource	Mascota	MascotaController, ReporteController, PanelRefugioController
SolicitudResource	Solicitud	SolicitudController, PanelRefugioController
SeguimientoResource	Seguimiento	SeguimientoController, SolicitudController, PanelRefugioController
ReporteResource	Reporte	ReporteController
SolicitudVoluntarioResource	SolicitudVoluntario	SolicitudVoluntarioController

2.17 Resumen de Códigos de Respuesta

Código	Cuándo
200	Operación exitosa (GET, PUT, PATCH, DELETE)
201	Recurso creado (POST)
400	Request inválida (ej: path traversal en archivos)
401	Token no presente o inválido/expirado
403	Sin permisos (rol insuficiente, recurso ajeno)
404	Recurso no encontrado

Código	Cuándo
422	Validación fallida o regla de negocio violada

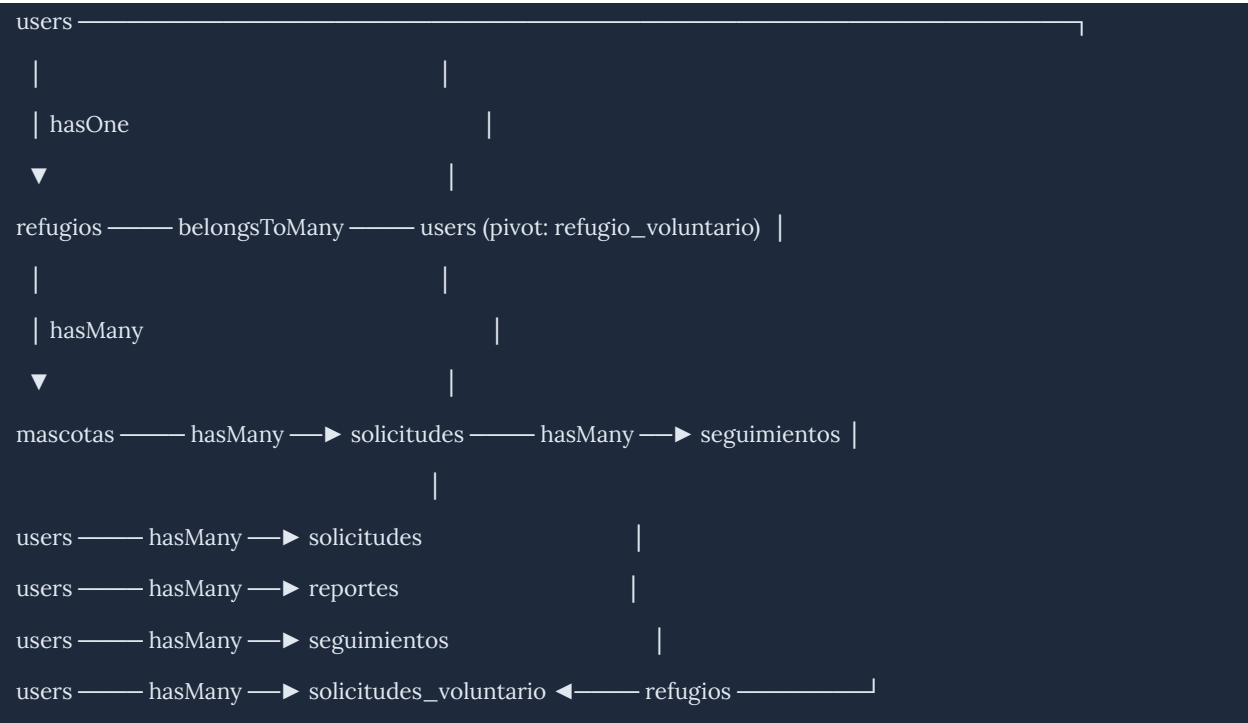
2.18 Matriz de Permisos por Endpoint

Endpoint	adoptante	voluntario	admin
GET /mascotas	✓	✓	✓
POST /mascotas	✗	✓	✓
POST /solicitudes	✓	✓	✓
PATCH /solicitudes/{id}/estado	✗	✓	✓
POST /reportes	✓	✓	✓
PATCH /reportes/{id}/estado	✗	✓	✓
POST /reportes/{id}/convertir-mascota	✗	✓	✓
POST /seguimientos	✓	✓	✓
PATCH /seguimientos/{id}/observacion	✗	✓	✓
GET /panel/*	✗	✓	✓
GET /users	✗	✗	✓
PATCH /users/{id}/role	✗	✗	✓
PATCH /users/{id}/estado	✗	✗	✓

3. Migraciones y Modelos

Stack: Laravel 13 · PostgreSQL · Spatie Permission · Firebase Auth

3.1 Diagrama de Relaciones



3.2 Enums (PHP 8.1 Backed Enums — string)

Especie

Caso	Valor
Perro	perro
Gato	gato

Sexo

Caso	Valor
Macho	macho
Hembra	hembra

Tamano

Caso	Valor
Pequeno	pequeno
Mediano	mediano
Grande	grande

EstadoMascota

Caso	Valor	Descripción
Disponible	disponible	Mascota lista para adopción
EnProceso	en_proceso	Solicitud activa pendiente de cierre
Adoptada	adoptada	Mascota ya adoptada

EstadoSolicitud

Caso	Valor	Descripción
Pendiente	pendiente	Recién enviada, sin revisar
EnRevision	en_revision	El refugio la está evaluando
Aprobada	aprobada	Solicitud aceptada (mascota → adoptada)
Rechazada	rechazada	Solicitud denegada

EstadoSeguimiento

Caso	Valor	Descripción
Pendiente	pendiente	El adoptante envió el seguimiento
Revisado	revisado	El refugio lo revisó sin observaciones
Observado	observado	El refugio añadió una observación

EstadoReporte

Caso	Valor	Descripción
Nuevo	nuevo	Reporte recién creado
EnAtencion	en_atencion	Un voluntario/admin lo tomó
Atendido	atendido	Animal o situación fue atendida
Cerrado	cerrado	Caso cerrado definitivamente

EstadoCuenta

Caso	Valor	Descripción
Activo	activo	Usuario puede operar con normalidad
Inactivo	inactivo	Cuenta deshabilitada temporalmente
Suspendido	suspendido	Cuenta suspendida por un admin

3.3 Tablas del Sistema (Laravel)

Generadas por las migraciones base de Laravel. No requieren modificación.

Tabla	Propósito
cache	Cache de aplicación
cache_locks	Locks de cache distribuido
jobs	Cola de trabajos en background

Tabla	Propósito
job_batches	Batches de jobs (procesamiento en lote)
failed_jobs	Jobs fallidos para reintento
password_reset_tokens	Tokens para recuperación de contraseña
sessions	Sesiones de usuario (si se usa driver DB)

3.4 Tabla: users

Migraciones: 0001_01_01_000000_create_users_table.php + 030309 + 030310

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
firebase_uid	varchar(255) UNIQUE	Sí	NULL	UID del usuario en Firebase Auth
name	varchar(255)	No	—	Nombre completo
email	varchar(255) UNIQUE	No	—	Correo electrónico
telefono	varchar(20)	Sí	NULL	Teléfono de contacto
direccion	varchar(255)	Sí	NULL	Dirección del usuario
foto_perfil_url	varchar(255)	Sí	NULL	URL de la foto de perfil
estado_cuenta	enum('activo','inactivo','suspendido')	No	activo	Estado de la cuenta
email_verified_at	timestamp	Sí	NULL	Fecha de verificación de email
password	varchar(255) hashed	Sí	NULL	Contraseña hasheada (nullable con Firebase)
remember_token	varchar(100)	Sí	NULL	Token de sesión persistente

Columna	Tipo	Nulable	Default	Descripción
created_at updated_at	/ timestamp	Sí	NULL	Timestamps automáticos

3.5 Tabla: refugios

Migración: 2026_06_16_030311_create_refugios_table.php

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
user_id	bigint unsigned FK → users.id	No	—	Admin/propietario del refugio
nombre	varchar(255)	No	—	Nombre del refugio
direccion	varchar(255)	No	—	Dirección física
telefono	varchar(20)	Sí	NULL	Teléfono de contacto
correo	varchar(255)	Sí	NULL	Correo del refugio
descripcion	text	Sí	NULL	Descripción/misión del refugio
logo_url	varchar(255)	Sí	NULL	URL del logo en almacenamiento
latitud	decimal(10,7)	Sí	NULL	Coordenada de latitud (GPS)
longitud	decimal(10,7)	Sí	NULL	Coordenada de longitud (GPS)
activo	boolean	No	true	Si el refugio aparece en listados públicos
created_at updated_at	/ timestamp	Sí	NULL	—

Foreign Keys: user_id → users.id CASCADE ON DELETE

3.6 Tabla: refugio_voluntario (Pivot)

Migración: 2026_06_21_155957_create_refugio_voluntario_table.php

Tabla pivot de la relación many-to-many entre refugios y sus voluntarios asignados.

Columna	Tipo	Descripción
refugio_id	bigint unsigned FK → refugios.id	Refugio al que pertenece
user_id	bigint unsigned FK → users.id	Voluntario asignado

Primary Key: compuesta (refugio_id, user_id) — Foreign Keys: ambas con CASCADE ON DELETE

3.7 Tabla: mascotas

Migración: 2026_06_16_030312_create_mascotas_table.php

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
refugio_id	bigint unsigned FK → refugios.id	No	—	Refugio al que pertenece
nombre	varchar(255)	No	—	Nombre de la mascota
especie	enum('perro','gato')	No	—	Especie (ver Especie)
raza	varchar(255)	Sí	NULL	Raza de la mascota
edad_meses	tinyint unsigned	Sí	NULL	Edad en meses (0-255)
sexo	enum('macho','hembra')	No	—	Sexo (ver Sexo)

Columna	Tipo	Nulable	Default	Descripción
tamano	enum('pequeno','mediano','grande')	Sí	NULL	Tamaño (ver Tamaño)
descripcion	text	Sí	NULL	Descripción general
personalidad	varchar(255)	Sí	NULL	Rasgos de personalidad
estado_salud	varchar(255)	Sí	NULL	Estado de salud actual
vacunas	boolean	No	false	Si tiene vacunas al día
esterilizado	boolean	No	false	Si está esterilizado/a
foto_url	varchar(255)	Sí	NULL	URL de la foto principal
fotos_extra	json	Sí	NULL	Array de URLs de fotos adicionales
estado_adopcion	enum('disponible','en_proceso','adoptada')	No	disponible	Estado actual de adopción
created_at / updated_at	timestamp	Sí	NULL	—

Foreign Keys: refugio_id → refugios.id CASCADE ON DELETE

3.8 Tabla: solicitudes

Migración: 2026_06_16_030313_create_solicitudes_table.php

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
user_id	bigint unsigned FK → users.id	No	—	Adoptante que envía la solicitud
mascota_id	bigint unsigned FK → mascotas.id	No	—	Mascota solicitada
refugio_id	bigint unsigned FK → refugios.id	No	—	Refugio que gestiona la solicitud
respuestas_formulario	json	No	—	Respuestas del formulario de adopción
estado_solicitud	enum('pendiente','en_revision','aprobada','rechazada')	No	pendiente	Estado actual
comentario_refugio	text	Sí	NULL	Feedback del refugio al adoptante
puntaje_evaluacion	tinyint unsigned	Sí	NULL	Puntaje asignado por el refugio (0-100)
created_at / updated_at	timestamp	Sí	NULL	—

Foreign Keys: user_id, mascota_id, refugio_id → todas con CASCADE ON DELETE

Lógica de Negocio en el Modelo

- `aprobar(?string $comentario) →` cambia `estado_solicitud` a `aprobada`, marca la mascota como adoptada y rechaza automáticamente todas las demás solicitudes pendientes/en revisión de esa mascota.
- `rechazar(?string $comentario) →` cambia `estado_solicitud` a `rechazada` y devuelve la mascota a disponible.

3.9 Tabla: seguimientos

Migración: `2026_06_16_030315_create_seguimientos_table.php`

Registros de seguimiento post-adopción que el adoptante envía periódicamente.

Columna	Tipo	Nulable	Default	Descripción
<code>id</code>	<code>bigint unsigned PK AI</code>	No	—	Identificador primario
<code>solicitud_id</code>	<code>bigint unsigned FK → solicitudes.id</code>	No	—	Solicitud de adopción relacionada
<code>mascota_id</code>	<code>bigint unsigned FK → mascotas.id</code>	No	—	Mascota adoptada
<code>user_id</code>	<code>bigint unsigned FK → users.id</code>	No	—	Adoptante que reporta
<code>foto_url</code>	<code>varchar(255)</code>	Sí	NULL	Foto actual de la mascota
<code>comentario</code>	<code>text</code>	Sí	NULL	Comentario del adoptante
<code>estado_mascota</code>	<code>varchar(255)</code>	Sí	NULL	Descripción libre del estado de salud actual
<code>observacion_refugio</code>	<code>text</code>	Sí	NULL	Observación del refugio sobre el reporte

Columna	Tipo	Nulable	Default	Descripción
estado_seguimiento	enum('pendiente','revisado','observado')	No	pendiente	Estado del seguimiento
created_at / updated_at	timestamp	Sí	NULL	—

Foreign Keys: solicitud_id, mascota_id, user_id → todas con CASCADE ON DELETE

3.10 Tabla: reportes

Migración: 2026_06_16_030314_create_reportes_table.php — Reportes SOS de animales en situación de calle o peligro.

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
user_id	bigint unsigned FK → users.id	No	—	Usuario que envía el reporte
descripcion	text	No	—	Descripción de la situación
foto_url	varchar(255)	Sí	NULL	Foto del animal/situación
latitud	decimal(10,7)	Sí	NULL	Latitud GPS del hallazgo
longitud	decimal(10,7)	Sí	NULL	Longitud GPS del hallazgo
direccion_referencia	varchar(255)	Sí	NULL	Dirección de referencia en texto
estado_reporte	enum('nuevo','en_atencion','atendido','cerrado')	No	nuevo	Estado del reporte

Columna	Tipo	Nulable	Default	Descripción
created_at updated_at	/ timestamp	Sí	NULL	—

Foreign Keys: user_id → users.id CASCADE ON DELETE

3.11 Tabla: solicitudes_voluntario

Migración: 2026_06_21_161444_create_solicitudes_voluntario_table.php

Solicitudes que un usuario envía para unirse como voluntario a un refugio.

Columna	Tipo	Nulable	Default	Descripción
id	bigint unsigned PK AI	No	—	Identificador primario
user_id	bigint unsigned FK → users.id	No	—	Usuario solicitante
refugio_id	bigint unsigned FK → refugios.id	No	—	Refugio al que solicita unirse
mensaje	text	Sí	NULL	Mensaje de motivación del solicitante
estado	enum('pendiente','aprobada','rechazada')	No	pendiente	Estado de la solicitud
comentario	text	Sí	NULL	Respuesta del refugio
created_at updated_at	/ timestamp	Sí	NULL	—

Índice único: (user_id, refugio_id) — un usuario solo puede tener una solicitud activa por refugio.

Foreign Keys: user_id, refugio_id → ambas con CASCADE ON DELETE

3.12 Tablas de Permisos (Spatie)

Migración: 2026_06_16_030308_create_permission_tables.php — Gestionadas automáticamente por spatie/laravel-permission.

Tabla	Descripción
permissions	Catálogo de permisos individuales
roles	Catálogo de roles (admin, voluntario, adoptante)
model_has_permissions	Permisos directos asignados a modelos
model_has_roles	Roles asignados a modelos (users)
role_has_permissions	Permisos asignados a roles

Roles del Sistema

Rol	Acceso
admin	Gestión completa: refugios, mascotas, solicitudes, reportes, usuarios
voluntario	Gestión de refugios asignados, mascotas, solicitudes y reportes
adoptante	Solo puede ver mascotas, enviar solicitudes, reportes y seguimientos

3.13 Modelos Eloquent

User — app/Models/User.php

Traits: HasFactory, Notifiable, HasRoles

Relación	Tipo	Modelo destino	Descripción
refugio()	HasOne	Refugio	El refugio que administra (si tiene uno)
refugiosComoVoluntario()	BelongsToMany	Refugio	Refugios donde es voluntario asignado

Relación	Tipo	Modelo destino	Descripción
refugiosGestionados()	Computed	Refugio	Unión de refugio propio + asignados
solicitudes()	HasMany	Solicitud	Solicitudes de adopción enviadas
reportes()	HasMany	Reporte	Reportes SOS enviados
seguimientos()	HasMany	Seguimiento	Seguimientos post-adopción enviados

Casts: email_verified_at → datetime, password → hashed, estado_cuenta → EstadoCuenta

Refugio — app/Models/Refugio.php

Relación	Tipo	Modelo destino	Descripción
admin()	BelongsTo	User	Propietario/admin del refugio
voluntarios()	BelongsToMany	User	Voluntarios asignados (pivot: refugio_voluntario)
mascotas()	HasMany	Mascota	Mascotas del refugio
solicitudes()	HasMany	Solicitud	Solicitudes recibidas

Método auxiliar: puedeGestionar(User \$user): bool — true si el user es el admin o un voluntario asignado.

Casts: activo → boolean, latitud/longitud → float

Mascota — app/Models/Mascota.php

Relación	Tipo	Modelo destino	Descripción
refugio()	BelongsTo	Refugio	Refugio al que pertenece
solicitudes()	HasMany	Solicitud	Solicitudes de adopción
seguimientos()	HasMany	Seguimiento	Seguimientos post-adopción

Método auxiliar: `estaDisponible(): bool` — true si `estado_adopcion === EstadoMascota::Disponible`.

Casts: `especie, sexo, tamano, estado_adopcion` → enums; `vacunas/esterilizado` → boolean; `fotos_extra` → array

Solicitud — `app/Models/Solicitud.php`

Relación	Tipo	Modelo destino	Descripción
<code>adoptante()</code>	BelongsTo	User	Usuario que envió la solicitud
<code>mascota()</code>	BelongsTo	Mascota	Mascota solicitada
<code>refugio()</code>	BelongsTo	Refugio	Refugio receptor
<code>seguimientos()</code>	HasMany	Seguimiento	Seguimientos asociados a esta adopción

- `aprobar(?string $comentario)` — aprueba la solicitud, marca la mascota como adoptada, rechaza automáticamente las demás solicitudes de la misma mascota.
- `rechazar(?string $comentario)` — rechaza la solicitud, devuelve la mascota a disponible.

Casts: `respuestas_formulario` → array; `estado_solicitud` → `EstadoSolicitud`

Seguimiento — `app/Models/Seguimiento.php`

Relación	Tipo	Modelo destino	Descripción
<code>solicitud()</code>	BelongsTo	Solicitud	Solicitud de adopción relacionada
<code>mascota()</code>	BelongsTo	Mascota	Mascota adoptada
<code>adoptante()</code>	BelongsTo	User	Usuario adoptante que reporta

Casts: `estado_seguimiento` → `EstadoSeguimiento`

Reporte — app/Models/Reporte.php

Relación	Tipo	Modelo destino	Descripción
reportador()	BelongsTo	User	Usuario que reportó

Casts: estado_reporte → EstadoReporte; latitud/longitud → float

SolicitudVoluntario — app/Models/SolicitudVoluntario.php

Relación	Tipo	Modelo destino	Descripción
solicitante()	BelongsTo	User	Usuario que solicita ser voluntario
refugio()	BelongsTo	Refugio	Refugio al que solicita unirse

Default attribute: estado = pendiente

3.14 Resumen de Relaciones

Tabla origen	Relación	Tabla destino	Via / FK
users	has one	refugios	refugios.user_id
users	belongs to many	refugios	refugio_voluntario
users	has many	solicitudes	solicitudes.user_id
users	has many	reportes	reportes.user_id
users	has many	seguimientos	seguimientos.user_id
users	has many	solicitudes_voluntario	solicitudes_voluntario.user_id
refugios	belongs to	users	refugios.user_id
refugios	belongs to many	users	refugio_voluntario
refugios	has many	mascotas	mascotas.refugio_id
refugios	has many	solicitudes	solicitudes.refugio_id

Tabla origen	Relación	Tabla destino	Via / FK
refugios	has many	solicitudes_voluntario	solicitudes_voluntario.refugio_id
mascotas	belongs to	refugios	mascotas.refugio_id
mascotas	has many	solicitudes	solicitudes.mascota_id
mascotas	has many	seguimientos	seguimientos.mascota_id
solicitudes	belongs to	users	solicitudes.user_id
solicitudes	belongs to	mascotas	solicitudes.mascota_id
solicitudes	belongs to	refugios	solicitudes.refugio_id
solicitudes	has many	seguimientos	seguimientos.solicitud_id
seguimientos	belongs to	solicitudes	seguimientos.solicitud_id
seguimientos	belongs to	mascotas	seguimientos.mascota_id
seguimientos	belongs to	users	seguimientos.user_id
reportes	belongs to	users	reportes.user_id
solicitudes_voluntario	belongs to	users	solicitudes_voluntario.user_id
solicitudes_voluntario	belongs to	refugios	solicitudes_voluntario.refugio_id